Engine · inline code

Inline code that reads on every surface

One chip rule, every surface — fill and border now derive from the ink, so the chip adapts instead of breaking.

The problem

A single `section code` rule, blind to its surface

The same `--accent` ink on a fixed `--bg-alt` fill — fine on the canvas, wrong everywhere else.

Where the old chip broke

- Dark bookends and the split-panel rail
  - They paint a dark surface without flipping `color-scheme`, so a light `--bg-alt` chip lands on dark
- Card and panel fills
  - A chip filled with `--bg-alt` vanishes into a `--bg-alt` card
- Accent-soft callouts
  - The gray chip fill clashes with the tinted `--accent-soft` panel
- Contrast was audited for the wrong pair
  - Themes measure `--accent` on `--bg`, but the chip rides `--bg-alt` — mustard fell to 3.89:1, below AA

The fix — a currentColor wash

- The chip ink is `--code-inline-fg`
  - Defaults to `--accent`; one value re-tunes the whole chip
- Fill and border derive from the ink
  - `color-mix(in srgb, currentColor 10%, transparent)` — always a subtle delta from the surface
- Surfaces rebind only the ink
  - Dark islands set `--code-inline-fg` to the on-dark tier; the wash follows

KEY INSIGHT

Rebind one token, not three. Because the fill tracks `currentColor`, setting the ink is enough — the chip stays legible whether it sits on a card, a callout, or a dark rail.

DARK RAIL

The chip on `--bg-dark`

A reference to `var(--accent)` on the dark rail now inks on-dark instead of flashing a near-white box.

> **Right zone keeps the canvas treatment**
>
> A `npm run build` reference reads as a soft accent chip on the light side

> **Same token, two surfaces**
>
> The rail rebinds `--code-inline-fg`; the right zone inherits the default

On tinted card fills

Canvas

A `var(--token)` chip is a faint accent wash

Card

The same `var(--token)` chip stays a delta from the `--bg-alt` fill

Callout

On `--accent-soft` the chip deepens instead of clashing

Dark

On a dark rail the ink flips and the wash follows

AA throughout

Inline code, in order

Surface-aware by construction — and ready for the interpreted-code proposal to build on.